SAP Application Development (S/4 HANA)

ABAP Objects

Motivation
This chapter explains the main concepts of objects-orientation and their adaption in ABAP.

Prerequisites
You should be familiar with ABAP and the navigation in the SAP system and Eclipse. Furthermore, you should have completed the *Introduction to SAP Programming* chapter.

Content
This chapter explains how to create simple programs in ABAP Objects.

Lecture notes
The fundamental understanding of the ABAP development in the SAP system is a prerequisite for the students. Students can go on with their account from previous chapters.

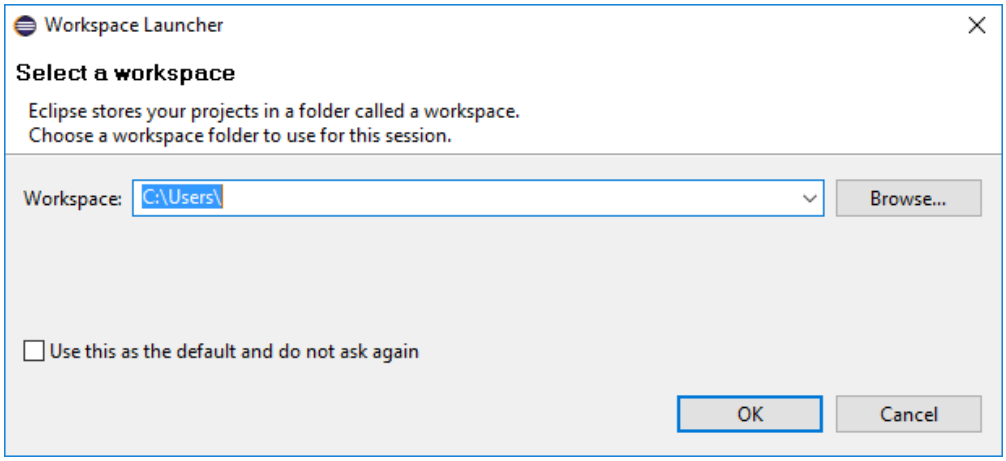
- **Product**: All
- **Level**: Advanced
- **Focus**: Programming
- **Version**: 2.0
- **Author**: UCC Technical University of Munich

Task 1: Login into the SAP system within Eclipse

Short description: Use Eclipse to login into the SAP system with your username and password

Login

Start Eclipse and select a local workspace where your ABAP project will be stored.



If you have already passed another learning module from this course, navigate to your project and login to the development system using your account from one of the Introduction modules (*Introduction to SAP Programming*, *Development of a first program*, ….).
Elsewise create a new ABAP Project and Package according to the instructions in the chapters *Introduction to SAP Programming* and *Development of a first program*.

Task 2: Working with ABAP Objects

Short description: Use the concept of ABAP Objects within an ABAP program – create a simple ABAP class.

Create a new ABAP program

To create a new ABAP Program right-click on your package and follow the path

NEW→ ABAP Program.

Replace ### with your group ID and $$ with your institution ID

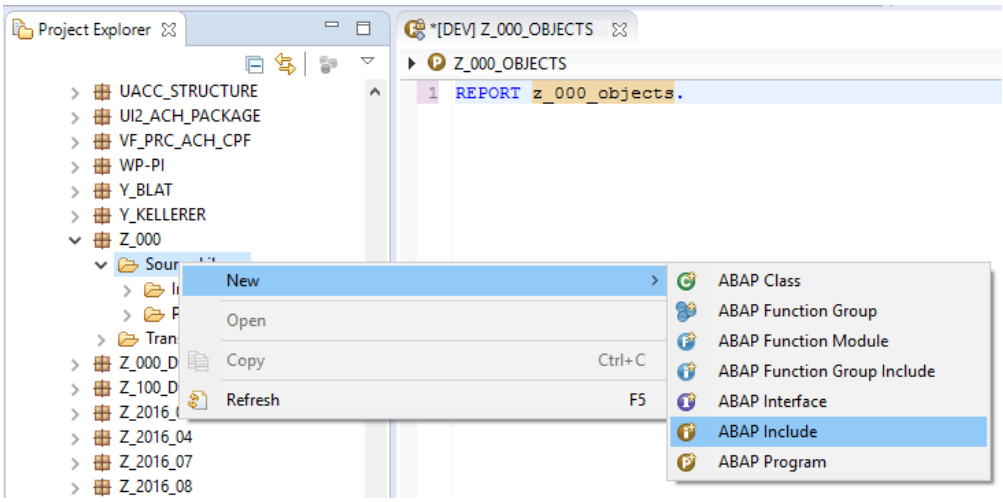
Name your ABAP Program '**Z_$$_###_OBJECTS**'.

Please always replace characters '###' with the group ID and $$ with the institution ID assigned to you.

You will create a new local class with the purpose of saving airlines. This class will be called in the program and several new instances of this class will be created.

Create a new include program **Z_$$_###_CLASS_INCLUDE**. For this, right-click the context menu of Source Library inside your package and select

New → ABAP Include.



Add command '**INCLUDE Z_$$_###_CLASS_INCLUDE.**' in your main program 'Z _$$_###_OBJECTS'. The newly created include program will include definition and implementation of the local class **LCL_AIRPLANE**.

Please start with the class declaration. You require a method **set_attributes** and a method **get_attributes** with public access. Method **set_attributes** includes an importing parameter **im_name** for the airplane name and importing parameter **im_planetype** for the airplane type. Both attributes have to be defined in the private section as attributes. The declaration of the local class is shown in the screenshot below.

Declare methods

Advice: Press Shift + Enter for code completion

Advice: Press Shift+F1 to run the Pretty Printer

```
CLASS lcl_airplane DEFINITION.
  PUBLIC SECTION.

  METHODS: set_attributes
                    IMPORTING
                      im_name      TYPE string
                      im_planetype TYPE string,
           get_attributes.

  PRIVATE SECTION.

  DATA: gv_name      TYPE string,
        gv_planetype TYPE string.

ENDCLASS.
```

Figure 1 - Z_$$_###_CLASS_INCLUDE

Method **set_attributes** should provide functionality to set both private attributes **name** and **planetype** according to the importing parameters. Method **get_attributes** should provide functionality to output the attribute values for a certain object with help of the WRITE-command. The implementation of both methods is shown here:

Implementation

```
CLASS lcl_airplane IMPLEMENTATION.

METHOD set_attributes.
  gv_name = im_name.
  gv_planetype = im_planetype.
ENDMETHOD.

METHOD get_attributes.
WRITE:  / 'Flight name: ', gv_name,
        / 'Flight type: ', gv_planetype.
ENDMETHOD.

ENDCLASS.
```

Figure 2 - Z_$$_###_CLASS_INCLUDE

The first screenshot illustrates the definition of the class, whereby the second one issues the implementation. Since they are connected to each other, they have to be put into one file.

Please save, check  and activate  your new program.

Task 3: Implement the use of a class

Short description: Implement the use of the ABAP class created in Task 2.

Please return to the main program in order to implement the use of the class. You will now create several airplanes (objects). The reference on these objects will be stored in an internal table in order to access these objects again at a later date.

The reference **r_plane** will use a new type definition: It will refer to the local class **LCL_AIRPLANE** by the use of declaration **TYPE REF TO**. In the same way the internal table **it_plane_list** is defined as table of references on this local class.

The data declaration of program '**Z_$$_###_OBJECTS**' therefore looks as shown in the figure below:

```
REPORT z_000_objects.

INCLUDE z_000_class_include.

DATA: r_plane TYPE REF TO lcl_airplane,
      it_plane_list TYPE TABLE OF REF TO lcl_airplane.
```

Figure 3 – Z_$$_###_OBJECTS

Now start the block '**START-OF-SELECTION**' by creating an object with command **CREATE OBJECT** that is referenced by variable **r_plane**.
Call method **set_attribute** using this reference and assign name and planetype to the new object using the corresponding parameters of the method.

Create two objects

Append the reference to table it_plane_list using the command **APPEND**. By appending the reference, the address of the created object is saved in the table and the reference **r_plane** itself can be used for other objects. Now create a second new airplane and append this reference to the internal tables as well (compare source code below).

Finally, you will loop at the internal table and display data for each airplane on the screen using method **get_attributes**.

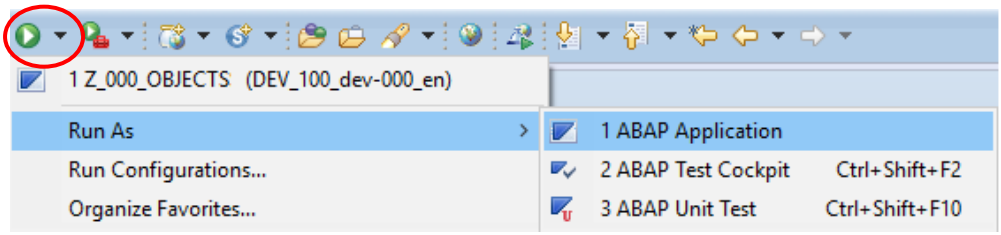
Loop through the list

```
START-OF-SELECTION.

CREATE OBJECT r_plane.
r_plane->set_attributes(
  EXPORTING
    im_name      = 'Hamburg'
    im_planetype = 'Boeing 737' ).
    APPEND r_plane TO it_plane_list.

    CREATE OBJECT r_plane.
r_plane->set_attributes(
  EXPORTING
    im_name      = 'Munich'
    im_planetype = 'Airbus 380' ).
    APPEND r_plane TO it_plane_list.

    LOOP AT it_plane_list INTO r_plane.
r_plane->get_attributes( ).
    ENDLOOP.
```

Figure 4 - Z_$$_###_OBJECTS

Please save, check and activate your new program.

Finally test your ABAP Application as shown in the screenshot below:



You should see the attributes of both created objects in the SAP GUI.

Task 4: Static components in ABAP Objects

Short description: Implement static components inside your ABAP class.

Until now, only instance attributes and methods have been implemented. Now, we want to count how many airplanes currently exist.

Create a static attribute in the private section of the declaration part of lcl_airplane called gv_no_of_planes. Select data type “Integer”.

Every time an airplane (object) is created, we want to add 1 to the number of airplanes. The method which is called every time an object is created, is the **constructor**

method. So, the coding to raise the number of airplane has to take place in the implementation part of this method. As all methods, the constructor method first needs to be declared in the declaration part of class lcl_airplane.

As the static attribute gv_no_of_planes was declared in the private section of class lcl_airplane, we can not access it directly via the command *lcl_airplane=>gv_no_of_airplanes.*

Though we need to implement another method get_no_of_planes exporting the number of airplanes.

```
CLASS lcl_airplane DEFINITION.
PUBLIC SECTION.

METHODS: set_attributes
                  IMPORTING
                    im_name TYPE string
                    im_planetype TYPE string,
         get_attributes,
         constructor.

CLASS-METHODS: get_no_of_planes EXPORTING no_of_planes TYPE i.

PRIVATE SECTION.

DATA: gv_name TYPE string,
      gv_planetype TYPE string.

CLASS-DATA gv_no_of_planes TYPE i.

ENDCLASS.
```

Figure 5 - Z_$$_###_CLASS_INCLUDE

```
CLASS lcl_airplane IMPLEMENTATION.

METHOD constructor.
gv_no_of_planes = gv_no_of_planes + 1.
ENDMETHOD.

METHOD get_no_of_planes.
no_of_planes = gv_no_of_planes.
ENDMETHOD.

METHOD set_attributes.
gv_name = im_name.
gv_planetype = im_planetype.
ENDMETHOD.

METHOD get_attributes.
WRITE: / 'Flight name: ', gv_name,
         'Flight type: ', gv_planetype.

ENDMETHOD.

ENDCLASS.
```

Figure 6 - Z_$$_###_CLASS_INCLUDE

Complete your program '**Z_$$_###_OBJECTS**' with the commands

```
DATA lv_no_of_planes TYPE i.

CALL METHOD lcl_airplane=>get_no_of_planes
  IMPORTING
    no_of_planes = lv_no_of_planes.
    WRITE: /  'Number of airplanes: ', lv_no_of_planes.
```

Figure 7 - Z_$$_###_OBJECTS

to display the number of airplanes on the screen.

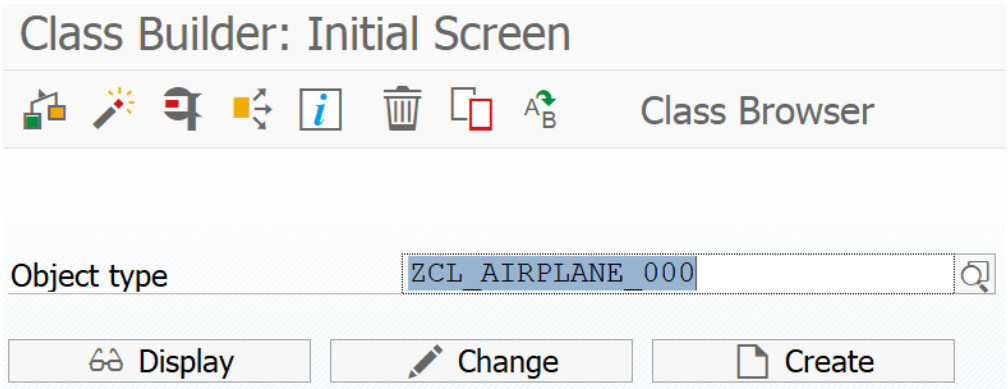
```
Z_WS16_00_OBJECTS
-------------------------------------------------------------------------------
Flight name:  Hamburg Flight type:  Boeing 737
Flight name:  Munich Flight type:  Airbus 380
Number of airplanes:           2
```

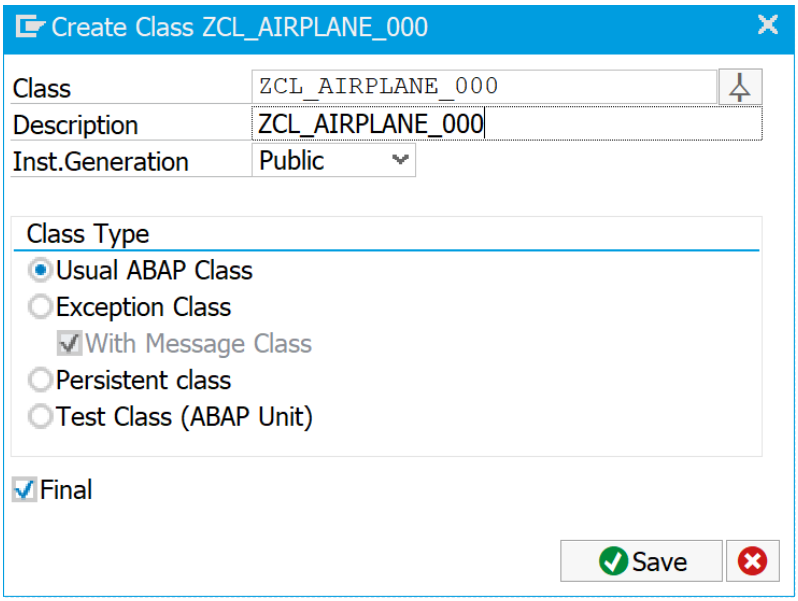
Challenge: Working with ABAP Objects and the Class Builder

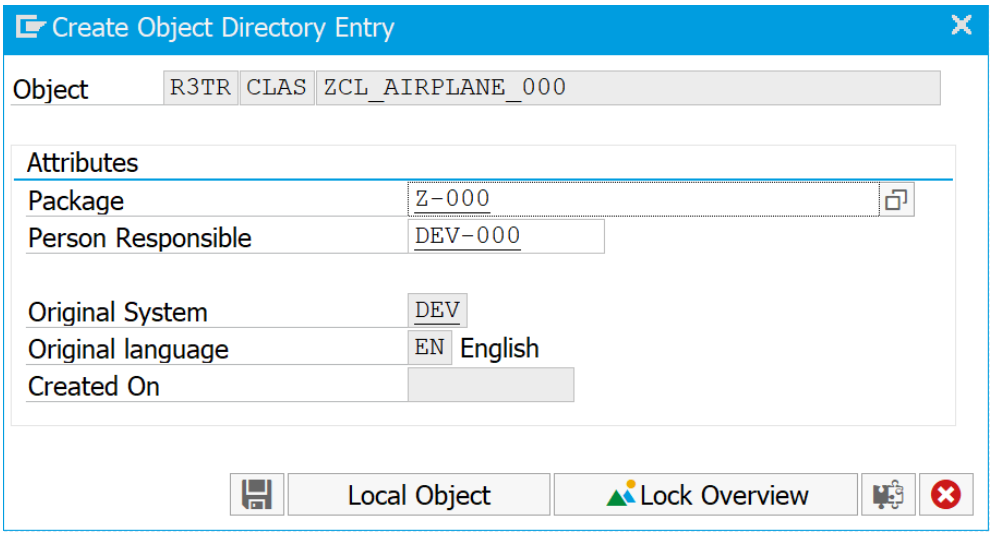
Short description: Implement tasks 3 and 4 within the Class Builder

Build class lcl_airplane in the class builder which you can access within the SAP GUI in transaction SE24. To navigate to the SAP GUI, select button ._

As with the Class Builder you can build global classes, please name it **'ZCL_$$_AIRPLANE_###'** and click on "**Create**".







Select Save and then Ok.

The Class Builder supports a very intuitive User Interface, try to implement all attributes and methods as described in tasks 2 and 3!

Create a report **'Z_$$_###_OBJECTS_GLOBAL'** in analogy to report **'Z_$$_###_OBJECTS'** with the following exceptions:

As we defined the class as "Public", it can be accessed from reports. So, you do not any more have to include your class with the help of an include statement into your report.

```
REPORT z_000_objects.

INCLUDE z_000_class_include.

DATA: r_plane TYPE REF TO lcl_airplane,
      it_plane_list TYPE TABLE OF REF TO lcl_airplane.
```

Furthermore, all occurrences of references to **lcl_airplane** have to be replaced with a reference to global class **zcl_$$_airplane**_###.

For experts: Why does the naming of every global class have to be unique (in this case assured by the addition '$$' and '_###' of your institution ID and group ID)?

Save, check, activate your program. You have done very well, if the output screen looks like in task number 4!